

Large Scale Social and Complex Networks: Design and Algorithms
ECE 232E — Summer 2026

Prof. Vwani Roychowdhury

UCLA Department of Electrical and Computer Engineering

Course Project

Structured Memory Networks and Reasoning Inference Chain Retrieval

Based on PANINI: Continual Learning in Token Space via Structured Memory

100 points

Due Saturday, August 15, 2026 at 11:59 PM Pacific Time

1 Purpose and learning objectives

Standard retrieval-augmented generation stores document chunks and asks a language model to repeatedly reason over retrieved text. PANINI instead converts documents into GSWs: heterogeneous semantic networks containing entities, verb-phrase/event nodes, roles, states, and grounded question-answer (QA) edges. At inference time, RICR decomposes a multi-hop query and searches for compact chains of QA pairs through this structured memory.

This project connects the PANINI framework to core topics from ECE 232E: heterogeneous graph construction, network projection, connectivity, centrality, ranking, sparse and dense search, beam search, scaling, and controlled experimental evaluation. By the end of the project, you should be able to:

1. construct and characterize corpus-level networks from document-level GSWs;
2. compare TF-IDF, BM25, dense, hybrid, and structure-aware retrieval;
3. execute and validate a neural question-decomposition plan;
4. implement RICR with chain scoring and diversity-preserving pruning;
5. analyze generalization from 2WikiMultiHopQA to MuSiQue; and
6. report accuracy, retrieval quality, latency, evidence size, and failure modes reproducibly.

2 Reading, repository, and compute

Required reading. Read Sections 1–4 and Algorithm 1 of [PANINI: Continual Learning in Token Space via Structured Memory](#). The GSW representation, dual indexing, decomposition notation, chain scoring, and RICR pruning rule in the paper are part of the project specification.

Student repository. The self-contained starter repository is:

<https://github.com/YigitTurali/panini-course-project>

The Colab notebook, artifact loaders, graph utilities, metrics, embeddings, indices, and model configuration are supplied. Students do **not** need access to the research codebase.

Models. Use the model identifiers in `models/model_config.json`. The required free-tier configuration uses:

- [GSW-QA-Decomposer-Qwen3-4B](#) for decomposition;
- supplied Qwen3-Embedding-8B corpus and fixed-query embeddings;
- [Qwen3-Reranker-8B](#) in 4-bit mode for reranking; and
- [Qwen3-4B](#) in 4-bit mode for evidence-grounded final answers.

The [8B decomposer](#) is optional and receives no automatic accuracy advantage. Do not keep the decomposer, reranker, query encoder, and answer model resident on the GPU simultaneously. Save outputs to JSONL, unload the current model, clear GPU memory, and then begin the next stage.

Compute policy. The required path must run on a free Google Colab GPU. No paid API is required or permitted for the required results. Corpus GSWs, embeddings, and indices are supplied; regenerating them is outside the project scope.

3 Datasets and evaluation protocol

You will receive two independent 100-question packages with the same directory contract and stable IDs.

Dataset	Development	Held out	Total	Required stratification
2WikiMultiHopQA	80	20	100	25 bridge-comparison, 25 comparison, 25 compositional, and 25 inference questions; each category has 20 development and 5 held-out questions.
MuSiQue	80	20	100	50 two-hop, 30 three-hop, and 20 four-hop questions; development/held-out counts are respectively 40/10, 24/6, and 16/4.

Table 1: Required course subsets.

- The **development split** includes questions, answers, aliases, supporting evidence, and context-document IDs. It may be used for debugging and tables in the report.
- The **held-out split** includes questions and document IDs but withholds answers and supporting evidence. Do not manually search for, reconstruct, or import these labels.
- Select algorithms and default hyperparameters using only the 2Wiki development split. Freeze them before running MuSiQue. MuSiQue is the required cross-dataset transfer evaluation.
- Run the final frozen system once on all 200 questions. Report metrics locally for the 160 development questions; the teaching staff will score the 40 held-out predictions.

MuSiQue is distributed under [CC BY 4.0](#); 2WikiMultiHopQA is distributed under [Apache 2.0](#). Retain the dataset IDs and attribution files in all derived outputs.

4 Fixed definitions and default settings

4.1 Graph views

For each GSW, construct the **native directed multigraph**. Entity and verb-phrase/event records are nodes. A grounded QA record is represented by directed, labeled edges that retain its stable QA ID,

question, answer entity, document ID, and relation to its verb-phrase. Do not merge records merely because their display strings match.

Also construct an **entity projection**. The supplied helper normalizes entity surface strings by case-folding, removing punctuation, and collapsing spaces. Two normalized entity names are adjacent when they answer QA records attached to the same verb-phrase. Store repeated co-occurrence as edge weight. This is a practical reconciliation rule, not a perfect entity linker. Count and discuss at least one possible false merge or missed merge. All component, clustering, and assortativity results must state whether they were computed on the native graph, its undirected view, or the entity projection.

4.2 Retrieval and RICR defaults

Unless a question explicitly requests an ablation, use:

Random seed	232
Dense similarity	inner product of L2-normalized vectors
Reciprocal Rank Fusion	$\text{RRF}(d) = \sum_r 1/(60 + r(d))$
Pre-rerank candidate pool	$M = 60$ unique QA records
Candidates returned per hop	$k = 15$
Beam width	$B = 5$
Chain score	geometric mean of hop scores
Generation	deterministic; sampling disabled

For a length- t chain $C = (e_1, \dots, e_t)$ with positive hop scores $s_1, \dots, s_t$, use

$$\text{score}(C) = \left(\prod_{\ell=1}^t \max(s_\ell, \epsilon) \right)^{1/t} = \exp \left(\frac{1}{t} \sum_{\ell=1}^t \log \max(s_\ell, \epsilon) \right), \quad \epsilon = 10^{-12}. \quad (1)$$

The log-domain form is recommended. At every pruning step, sort by decreasing chain score and use the stable QA-ID sequence as the deterministic tie-breaker. Retain at most one chain per normalized *current answer*.

4.3 Metrics

Use the provided normalization functions. Report:

Retrieval: Recall@ k for $k \in \{1, 5, 10, 15\}$, reciprocal rank/MRR, supporting-document recall, and supporting-QA recall.

Chains: complete-chain recovery (all required supporting hops occur in one surviving chain), average surviving chains, and average unique current answers.

Answers: Exact Match (EM) and token F1, giving each question the maximum score over its accepted answer aliases.

Efficiency: wall-clock latency per stage, peak GPU memory when a model is used, number of reranked candidates, number of returned QA pairs, and answer-model input tokens.

Aggregate by macro-average over questions. Include the number of evaluated questions in every table. Never average already-rounded subgroup values.

5 How to build the system

Build and test the project in the order below. Each stage has a small, inspectable output that becomes the input to the next stage; do not begin with the complete end-to-end pipeline.

5.1 Load records and preserve stable identifiers

Use `CoursePackage` to load JSONL records and the supplied matrices. Represent graph nodes and retrieval candidates with their stable `entity_uid` and `qa_uid`; never use a list position as an identifier. Keep document ID, node type, edge direction, role/state text, question, answer, and source retrieval score attached to the record. Before building an index, assert that every matrix row has exactly one ID in its matching ID file and that all IDs are unique.

5.2 Construct the two graph views

For the native directed graph, namespace node keys by node type so that an entity and a QA record cannot collide. Add an entity node for each entity record and a QA node for each QA record. Add the directed verb edge from its source entity to the QA node and the directed answer edge from the QA node to its answer entity. Preserve the originating document ID on nodes and edges.

For the entity projection, first collect the set of normalized entity names that occur in each document. For every unordered pair in that set, add one to the pair’s co-occurrence weight. Remove self-pairs. Thus an edge weight is the number of documents in which the two normalized entities co-occur, not the number of times that their strings appear. Test this construction on a three-entity hand-written document before processing the supplied corpus.

5.3 Give every retriever the same interface

Each retriever should return ordered records of the form `(qa_uid, score, rank, source)`. This makes the following methods directly comparable.

1. Fit TF-IDF and BM25 only to the supplied QA question strings. Transform a query with the same tokenizer and return the highest-scoring QA IDs.
2. For dense retrieval, L2-normalize the query vector and use inner product against the supplied normalized QA vectors.
3. For entity expansion, retrieve entities first and gather the QA IDs adjacent to those entities in the native graph.
4. For RRF, assign each candidate its best rank in every source and sum $1/(60 + r)$. A candidate absent from a source contributes zero.
5. For dual retrieval, take the union of direct QA candidates and entity-expanded QA candidates, deduplicate by `qa_uid`, truncate to the fixed $M = 60$ pool, and only then apply the reranker.

Write a toy corpus with one obviously relevant QA record and verify the rank returned by every backend. This catches ID-order and score-direction errors before they affect the full experiment.

5.4 Parse the decomposition into an executable dependency graph

Parse every generated line into a template ID and text. A placeholder such as `<ENTITY_Q1>` creates a dependency on the answer produced by Q1. Reject duplicate template IDs, references to missing or future steps, and unresolved placeholders. A template that calls for retrieval creates a retrieval node; comparison, intersection, or other deterministic operations create operation nodes. Templates with no dependency on one another form parallel sequences. Topologically sort the graph and save both the raw model text and parsed structure so parsing failures can be inspected without rerunning the model.

5.5 Implement RICR with a toy retriever first

A chain contains an immutable ordered tuple of evidence records, the current answer, the instantiated sub-question at each hop, and the log scores seen so far. Implement the linear search in this order:

```
beams = retrieve the first instantiated sub-question
beams = keep the best B distinct normalized answers
for each remaining template:
    expansions = []
    for chain in beams:
        query = substitute chain.current_answer into the template
        for candidate in retrieve(query, k):
            expansions.append(a new chain; do not mutate chain)
    score every expansion by its log-domain geometric mean
    beams = globally keep the best B distinct normalized answers
return beams and the complete trace
```

The pruning input contains as many as kB expansions. Sort it once by decreasing chain score, then by the tuple of stable QA IDs. Scan that order and keep the first chain for each normalized current answer until B remain. Do not prune independently inside each parent beam. When parallel sequences have finished, merge their evidence by stable QA ID and retain the provenance from the highest-scoring occurrence.

First run this logic with a dictionary-backed toy retriever whose outputs you control. After the two-hop, duplicate-answer, tie, empty-result, and $B = 1$ cases pass, replace the toy retriever with the real retrieval interface. Run `pytest -q tests`; the supplied RICR tests are initially skipped while the two functions contain `NotImplementedError` and become active automatically after implementation.

6 Graded questions

6.1 Question 1: Understand the two data packages [4 points]

Load both supplied packages before implementing an algorithm.

- Produce a table containing, for each dataset and split, the number of questions, unique documents, GSW files, entities, QA records, and decomposition-validation examples.
- Show one question, entity, and QA record. Explain the stable ID and every field that your implementation consumes.
- Add an assertion that held-out records contain no answer-label fields: `answer`, `answer_aliases`, `supporting_facts`, or `evidences`.
- Run `pytest -q tests`. Record which tests pass and which RICR tests are skipped before you begin the implementation.

Required output. One dataset table, three short schema examples, the no-leakage assertion, and the initial test summary in the notebook.

Point allocation. Dataset table (1), schema explanation (1), automated leakage check (1), and starter-test run (1).

6.2 Question 2: Construct the native GSW network and entity projection [8 points]

Implement a loader that combines the document-level GSWs into the two graph views defined above. Preserve node type, document provenance, role/state text, QA ID, and direction in the native graph. For the projection, use the supplied surface-name normalization helper and document its limitations.

- (a) Report native-graph node and edge counts by node/edge type and verify them against direct counts from the JSON files.
- (b) Construct the weighted entity projection and report its node count, edge count, density, number of self-loops removed, and total edge weight.
- (c) Add at least three small unit tests: one for direction, one for document provenance, and one for projection edge weights.

Required output. Construction code, a graph-statistics table for each dataset, and passing tests.

Point allocation. Correct native graph (3), correct projection/reconciliation (3), validation table (1), and tests (1).

6.3 Question 3: Analyze the structured-memory network [8 points]

Perform the following analysis separately for 2Wiki and MuSiQue:

- (a) Compute weakly connected components of the native graph and connected components of the entity projection. Report the number of components, giant-component size and fraction, isolated nodes, and component-size summary (minimum, median, mean, and maximum).
- (b) Plot degree probability mass and complementary cumulative degree distributions on log-log axes. State whether the heavy tail, if any, is consistent across datasets; do not claim a power law without a fitted test.
- (c) On the entity projection, report the ten highest nodes under weighted degree, PageRank, and betweenness centrality. Explain at least one disagreement between rankings. Exact betweenness or a documented fixed-seed approximation is acceptable.
- (d) Report average clustering and degree assortativity, then visualize one complete gold multi-hop path from a development question with node types and edge/QA labels.

Required output. At least four plots, two summary tables, and a 150–250 word cross-dataset interpretation.

Point allocation. Connectivity (2), degree analysis (2), centrality (2), and clustering/assortativity/path visualization (2).

6.4 Question 4: Question decomposition and dependency graphs [10 points]

Run the supplied 4B decomposer on all 200 questions using the supplied prompt, sampling disabled, and the model-config maximum output length. Cache one JSONL record per question. Parse numbered sub-questions and <ENTITY_Qn> placeholders into a directed acyclic dependency graph. Retrieval steps and deterministic reasoning-only steps (for example, comparing two dates) must be marked separately.

- (a) Validate references: every placeholder must point to an earlier sub-question, the graph must be acyclic, and every retrieval node must contain a nonempty natural-language question.

- (b) Identify independent linear sequences that may run in parallel and record the deterministic operations needed to combine their answers.
- (c) On the public reviewed decomposition subset, report valid-plan rate, sub-question-count exact match, dependency-edge precision/recall/F1, and retrieval-vs-reasoning node accuracy.
- (d) Analyze two errors from each dataset. Distinguish malformed output, wrong dependency, missing hop, extra hop, and semantically incorrect atomic question.

Required output. `decompositions.jsonl`, parser tests, a metric table, and four concise error analyses.

Point allocation. Inference and caching (2), parser/validation (3), parallel-sequence handling (2), metrics (2), and error analysis (1).

6.5 Question 5: Sparse retrieval baselines

[8 points]

Using only the supplied metadata, implement and evaluate:

- (a) TF-IDF retrieval over concatenated QA question and answer text;
- (b) BM25 retrieval over the same QA text; and
- (c) BM25 entity retrieval over surface name, aliases, roles, and states, followed by expansion to QA records attached to each retrieved entity.

Use identical text normalization within a method across datasets. State all tokenization and BM25 parameters. Evaluate the gold atomic sub-questions from the development splits at $k \in \{1, 5, 10, 15\}$. Report Recall@ k and MRR overall, by 2Wiki question type, and by MuSiQue hop count.

Required output. One implementation per method, a complete retrieval table, and a short explanation of where entity expansion helps or hurts.

Point allocation. Correct TF-IDF/BM25 QA search (3), correct entity expansion (2), evaluation (2), and interpretation (1).

6.6 Question 6: Dense, hybrid, and paper-style dual retrieval

[12 points]

Load the supplied normalized QA embeddings and FAISS index; do not regenerate corpus embeddings. Use the supplied fixed-query embedding when present. If answer substitution creates a new query, encode it with the configured query encoder and cache it by exact normalized query string.

- (a) Implement dense QA retrieval and verify that a manual inner-product calculation agrees with FAISS for at least five queries.
- (b) Implement RRF over TF-IDF, BM25-QA, and dense rankings with constant 60. Deduplicate by stable QA ID.
- (c) Implement paper-style dual retrieval: union QA records obtained from BM25 entity expansion and dense QA search, deduplicate to a pre-rerank pool of at most $M = 60$, rerank against the instantiated atomic question, and return the top $k = 15$.
- (d) Compare all methods using the Question 5 protocol. Add mean and 95th-percentile retrieval latency and the average candidate count.

Required output. Search code, FAISS consistency test, performance/latency tables, and the saved top-15 retrieval trace for every development atomic question.

Point allocation. Dense retrieval (3), RRF (2), dual candidate construction (3), reranking/deduplication (2), and evaluation (2).

6.7 Question 7: Does reranking always help? [5 points]

Compare three scoring rules on the same $M = 60$ candidate pool:

- (i) reranker probability only;
- (ii) reciprocal retrieval rank only; and
- (iii) 0.5 reranker probability + 0.5 reciprocal retrieval rank.

Use rank reciprocal $1/r$ after assigning each candidate its best pre-rerank rank. Report Recall@15 and MRR for each dataset. Find one query for which reranking improves the first relevant rank and one for which it worsens it. For each, show the top five candidates before and after scoring and explain the lexical/entity mismatch.

Required output. One comparison table and two annotated retrieval traces.

Point allocation. Controlled comparison (2), traces (2), and evidence-based explanation (1).

6.8 Question 8: Implement Reasoning Inference Chain Retrieval [15 points]

Complete `prune_unique_answers` and `run_linear_ricr` in `panini_course/ricr.py`. For each linear sequence:

- (1) retrieve k candidates for the first instantiated sub-question;
- (2) retain the top B chains with distinct normalized current answers;
- (3) substitute each current answer into the next dependent sub-question;
- (4) expand all surviving chains by k candidates;
- (5) update the geometric-mean chain score in the log domain;
- (6) globally prune the kB expansions to B distinct current answers;
- (7) repeat until the sequence ends; and
- (8) combine parallel sequences by deduplicating evidence using stable QA ID while preserving the highest-scoring provenance.

A candidate from an earlier chain may not be mutated in place. Ties must use the deterministic rule from Section 4.2. Return both the deduplicated evidence and complete trace: instantiated questions, candidates, scores, pruned chains, and selected QA IDs at every hop.

Required output. Working implementation, all supplied RICR tests passing, at least three additional tests of your own (including parallel sequences), plus one human-readable trace from each dataset.

Point allocation. First-hop/pruning behavior (3), multi-hop expansion/substitution (4), chain scoring (2), parallel evidence merge (2), deterministic trace format (1), and tests/edge cases (3).

6.9 Question 9: RICR ablation study**[10 points]**

Use a fixed, stratified 20-question subset from each development dataset for one-factor-at-a-time ablations. Publish the selected IDs. The default is $B = 5$, $k = 15$, unique-answer pruning enabled, geometric-mean scoring, and the best frozen Question 7 retrieval score.

- (a) Beam width: $B \in \{1, 3, 5\}$.
- (b) Candidates per hop: $k \in \{5, 15\}$.
- (c) Diversity: unique-answer pruning on versus off.
- (d) Chain scoring: geometric mean versus last-hop score.
- (e) Retrieval: BM25-QA, dense QA, RRF hybrid, and paper-style dual retrieval.

Report supporting-QA recall, complete-chain recovery, answer EM/F1, mean latency, and mean evidence count. Plot accuracy versus latency and accuracy versus evidence size. Change only the named factor in each comparison.

Required output. One ablation table, two trade-off plots, and a 200–300 word recommendation of the configuration you would deploy.

Point allocation. Experimental control (2), beam/candidate ablations (2), diversity/scoring ablations (2), retrieval ablation (2), and trade-off analysis (2).

6.10 Question 10: End-to-end 2Wiki evaluation**[7 points]**

Freeze one final configuration using the 2Wiki development split and run it on all 100 2Wiki questions. Generate final answers using only the deduplicated QA evidence returned by RICR; the answering prompt must instruct the model to return N/A when evidence is insufficient.

On the 80 labeled questions, report decomposition validity, supporting-QA recall, complete-chain recovery, EM, F1, average/95th-percentile latency, answer-context tokens, and evidence count. Break out EM/F1 by the four 2Wiki question types. Include one successful and one failed complete trace.

Required output. `results/2wiki_dev.jsonl`, `predictions/2wiki_heldout.jsonl`, one main table, one type table, and two traces.

Point allocation. Correct frozen run/output files (2), complete metrics (2), type-specific analysis (1), trace quality (1), and grounded answer prompting (1).

6.11 Question 11: MuSiQue transfer and scaling evaluation**[8 points]**

Without changing retrieval weights, M , k , B , prompts, or score rules, replace the 2Wiki package with the MuSiQue package and run all 100 questions. On the 80 labeled questions:

- (a) report the same metrics as Question 10, separated into 2-, 3-, and 4-hop groups;
- (b) plot complete-chain recovery and F1 against hop count;
- (c) compare 2Wiki and MuSiQue under the same frozen configuration; and
- (d) identify whether the largest transfer loss comes from decomposition, first-hop retrieval, later-hop substitution/retrieval, or answer generation, using counts from saved traces rather than intuition.

Required output. Submit:

- `results/musique_dev.jsonl` and `predictions/musique_heldout.jsonl`;
- a hop-count table and two scaling plots; and
- a quantitative transfer-error analysis.

Point allocation. Frozen transfer run (2), hop-stratified metrics (2), scaling plots (1), cross-dataset comparison (1), and traced error attribution (2).

6.12 Question 12: Reproducibility and final submission

[5 points]

Submit one archive or course-repository commit containing:

- (a) `report.pdf`, organized by Questions 1–12 and limited to 12 pages excluding references and a one-page appendix;
- (b) the completed Colab notebook and any Python modules/tests it imports;
- (c) all four development/held-out JSONL files from Questions 10–11;
- (d) `environment.txt` with Python, CUDA, package, and model revisions; and
- (e) `RUNME.md` with exact commands, seeds, expected runtime, restart points, and the final Git commit hash.

The notebook must run from a fresh Colab runtime using relative paths after setting one `PACKAGE_ROOT`. Do not submit downloaded model weights, provided embeddings, FAISS indices, or dataset copies.

Point allocation. Report organization/completeness (2), executable code/tests (1), required result schemas (1), and environment/RUNME reproducibility (1).

7 Required result schema

Each prediction file must contain one JSON object per line and preserve the input order. Extra diagnostic fields are allowed, but the following fields are required:

```
{
  "dataset": "2wiki" | "musique",
  "split": "development" | "held_out",
  "question_id": "...",
  "question": "...",
  "predicted_decomposition": [...],
  "decomposition_valid": true,
  "retrieval_backend": "dual_hybrid",
  "beam_width": 5,
  "candidates_per_hop": 15,
  "chains": [
    {
      "qa_ids": ["...", "..."],
      "answers": ["...", "..."],
      "hop_scores": [0.91, 0.87],
      "chain_score": 0.8898
    }
  ],
  "evidence_qa_ids": ["...", "..."],
```

```

"predicted_answer": "...",
"latency_ms": {
  "decomposition": 0.0,
  "retrieval_ricr": 0.0,
  "answer": 0.0
},
"answer_context_tokens": 0
}

```

Development files may additionally contain gold labels and computed metrics. Held-out files must not contain gold fields.

8 Grading summary

Component	Points
Q1. Understand the two data packages	4
Q2. Native GSW network and entity projection	8
Q3. Structured-memory network analysis	8
Q4. Question decomposition and dependency graphs	10
Q5. Sparse retrieval baselines	8
Q6. Dense, hybrid, and dual retrieval	12
Q7. Reranking analysis	5
Q8. RICR implementation	15
Q9. RICR ablations	10
Q10. 2Wiki end-to-end evaluation	7
Q11. MuSiQue transfer and scaling	8
Q12. Reproducibility and submission	5
Total	100

9 General grading and conduct

- A table or plot without axis labels, units, evaluated-question count, or a caption receives at most half credit for that item.
- Results that cannot be reproduced from the submitted code receive no implementation credit, even if the reported values are plausible.
- Clearly label any deviation from the required settings. An additional experiment does not replace a required one.
- You may use standard libraries for graph storage, TF-IDF, BM25, FAISS, plotting, and model loading. You may not import an implementation of RICR or copy an answer key.
- Follow the course collaboration and academic-integrity policy. Cite all external code, prompts, and written sources.
- The course late-submission policy applies. If a released artifact is corrupted or a Colab dependency becomes unavailable, post a minimal reproducer to the course forum before the deadline.

The goal is not merely to obtain a final answer. Your system should make the network path, evidence, score, and failure mode inspectable.